

UNITAT 1 · SETMANA 1

Introducció als sistemes operatius

📅 Setmana 1 · dates per confirmar </> Laboratori TD0 (resolt + repte IA) ~ 4 animacions

LABORATORI TD0 · RESULT

Preparar l'entorn de programació de sistemes

Abans de programar contra el sistema operatiu necessites quatre coses: saber llegir el manual, un esquelet de programa, una petita biblioteca d'utilitats i un `Makefile` que ho compili tot convertint els avisos en errors.

</> Com fer servir els laboratoris resolt

Cada laboratori inclou la solució completa. El codi que veus aquí és exactament el de la carpeta `tds/` del curs, compilat amb `-Wall -Wextra -Werror` i provat per `tds/tests/run.sh`; les sortides surten d'una execució real. La teva feina: compilar, executar, canviar alguna cosa, predir què passarà i comprovar-ho. **Sempre compiles amb gcc**, amb els avisos convertits en errors: aquesta és la línia de cada programa (el `Makefile` fa servir exactament la mateixa per compilar-los tots de cop).

Bash

```
cd tds/td0
gcc -std=gnu11 -O2 -Wall -Wextra -Werror -D_GNU_SOURCE -pthread -I../common
errors.c ../common/utils.c -o errors -lm
./errors

make -C tds          # the same gcc line for every program of every lab
(make prints each one)
tds/tests/run.sh td0 # tests one lab and prepares its working folder
tds/tests/tmp/td0
```

Opcional: el terminal dins de l'aula

Pots treballar els laboratoris amb un **terminal real de la teva màquina** dins d'aquesta pàgina: cada sortida desada que es pot repetir té un botó ► **Executa** que executa la mateixa ordre a la teva màquina, i el botó ► **_ Terminal** (a baix a la dreta, a les pàgines de laboratori) obre un terminal on escrius el que vulguis. Un navegador no pot iniciar processos tot sol, així que un petit programa del curs fa de pont.

Bash

```
make -C tds && tds/tests/run.sh # once: builds every lab and prepares
tds/tests/tmp/td0 ... td8
python3 tds/terminal.py # leave this terminal open while you work
```

El pont mostra un **enllaç** (obre'l i l'aula es connecta sola) i una **clau** com `8765.1a2b...`, que també pots enganxar al tauler. La clau canvia cada cop que l'inicies. **Ctrl+C** el tanca, juntament amb tots els seus terminals.

- **Sempre amb gcc** : ► **Executa** escriu, una ordre per línia, el `cd` a la carpeta del laboratori, un `gcc` per cada programa de l'ordre (l'executable va a la carpeta de treball, `tds/tests/tmp/td3`) i l'ordre de la sortida desada, que s'executa allà, al costat dels fitxers de prova. Si canvies el codi i el tornes a prémer, en veus l'efecte; si `gcc` dona un error, no s'executa res. ⚙️ **Compila amb gcc**, al codi de cada exercici, només compila (amb totes les seves variants `-D...`). Si el laboratori encara no té carpeta de treball, primer la prepara amb `tests/run.sh`; **Prepara TDn** fa només això. Compara la teva sortida amb la desada: els PID, les adreces i els temps canvien, i explicar per què forma part del laboratori.
- **Finestres flotants**: cada terminal flota sobre la pàgina mentre llegeixes l'enunciat, el manual o les animacions. Mou-lo arrossegant-ne la barra (o amb les fletxes quan la barra té el focus), canvia'n la mida des de la cantonada i minimitza'l amb `-`; recorda on l'has deixat. `+` n'obre un altre (fins a 3), pràctic per a un servidor i el seu client (TD2, TD7); ► **Executa** escriu a l'últim que has fet servir. 🗑️ el passa a una finestra separada, sempre al davant fins i tot d'altres aplicacions, com el PDF del laboratori (Firefox i Chrome recents); en tancar-la torna a la pàgina.
- **Ubuntu**: inicia'l al Terminal (Ctrl+Alt+T), dins de la carpeta del curs. Funcionen tant el Firefox que ve amb Ubuntu com Chrome. **Windows**: inicia'l dins de WSL 2 (el terminal d'Ubuntu), on compiles els laboratoris; el navegador de Windows arriba al `127.0.0.1` de WSL perquè WSL redirigeix `localhost` per defecte.
- **També al navegador**: el pont serveix l'aula a `http://127.0.0.1:8765/` (el segon enllaç que mostra, amb la clau `aquí...`). És còmode a Windows amb WSL: obre'l al navegador de Windows.
- **Sense Ubuntu: GitHub Codespaces**. Al repositori del curs, `github.com/andresrubiop/fso-uti`, prem *Code* → *Codespaces*: tindràs un Ubuntu 26.04

al núvol amb els laboratoris ja compilats i un terminal al navegador. Executa-hi `python3 tds/terminal.py` i obre l'enllaç `https://...app.github.dev/?terminal=aquí...` que mostra: és l'aula amb el terminal connectat al Codespace. El pont només accepta l'adreça privada d'aquell Codespace; deixa el port 8765 privat. Fa servir la quota gratuïta del teu compte de GitHub: atura el codespace quan acabis. Les mesures (memòria, temps, Landlock del TD8) poden ser diferents de les d'un Ubuntu instal·lat.

- **Seguretat:** un shell a l'abast del navegador és delicat, per això el pont només escolta a `127.0.0.1` (mai a la xarxa), exigeix la clau d'aquella sessió, rebutja pàgines d'altres llocs, obre com a màxim 3 terminals i tanca cadascun quan tanques la pestanya. És el teu shell, amb els teus permisos: no enganxis ordres que no entenguis.

Sense el pont tot continua igual: copia les ordres al teu terminal. Les sortides que necessiten un servidor a part (TD7) o el dimoni `agentd` (TD8) no tenen botó; els passos són a l'enunciat.

L'informe de cada laboratori, en LaTeX

Des del TD0, cada laboratori s'entrega amb un **informe curt en LaTeX** amb el format del curs: el just i necessari, de 2 a 4 pàgines. Per a cada exercici: què has fet i per què, només el codi que importa, el que has executat al terminal (tal qual) i una conclusió d'una o dues línies. Entregues el PDF i el `.zip` del projecte.

La plantilla porta el format de la UTI (logos, colors, capçalera) i tres entorns: `terminal` per enganxar tal qual el que has fet al terminal (com *verbatim*), `codigo` per al codi en C amb colors i `declaracionIA` per declarar l'ús d'IA.

A Prism (OpenAI), pas a pas

1. Entra a prism.openai.com amb el teu compte de ChatGPT: és gratuït.
2. Prem **Import** i tria `FS0_plantilla_informe_LaTeX.zip` (o arrossega'l a la llista de projectes): Prism crea el projecte amb la plantilla.
3. Obre `informe.tex`, posa-hi les teves dades (`\td`, `\estudiante`, `\paralelo`) i compila per veure el PDF.
4. Escriu cada exercici amb `\ejercicio{...}`. El que has fet al terminal va entre `\begin{terminal}` i `\end{terminal}`: es copia tal qual. El codi, entre `\begin{codigo}` i `\end{codigo}`.
5. Descarrega el PDF i el projecte (`.zip`) i entrega'ls a Canvas (Portafoli · TDn).

IA generativa: permesa en tot moment

Pots fer servir l'assistent de Prism (o qualsevol IA) per crear el LaTeX: donar format, corregir errors de compilació, ordenar taules o millorar la redacció. Declara-ho al requadre final (eina, per a què i com ho has verificat). Les sortides han de ser les del teu terminal i has de poder explicar cada línia: això és el que s'avalua.

Sense Prism, la plantilla compila igual a Overleaf o al teu Ubuntu:

```
sudo apt install texlive-latex-extra texlive-lang-spanish latexmk i després
latexmk -pdf informe.tex.
```

EXERCICIS

- | | | | |
|---|--|---|---------------------------------|
| 1 | Pàgines i seccions del manual | 4 | Compilar amb <code>make</code> |
| 2 | L'esquelet de programa | 5 | Explora el teu sistema operatiu |
| 3 | La biblioteca <code>libutils</code> i <code>exit_if</code> | | |

TD0 · 1

● Result

Pàgines i seccions del manual

Enunciat. En aquest curs escrivim `cmd(k)` per dir que `cmd` està documentat a la secció `k` del manual. Fes servir `man -k` per cercar per paraula clau i respon: a quina secció és el `printf` de la biblioteca de C? Quines són les altres pàgines que apareixen? Quina diferència hi ha entre les seccions `3` i `3p`?

Solució

Bash

```
man -k printf | grep ^printf      # keyword search, keep the names that start
with printf
man 3 printf                      # the C library function
man 1 printf                      # the shell command with the same name
man 3p printf                     # the POSIX standard text (package
manpages-posix-dev)
```

SECCIÓ	CONTINGUT	EXEMPLE
1	Programes i ordres del shell	<code>ls(1)</code> , <code>printf(1)</code>
2	Crides al sistema (entrades al nucli)	<code>write(2)</code> , <code>fork(2)</code>
3	Funcions de biblioteca (glibc a Linux)	<code>printf(3)</code> , <code>perror(3)</code>
3p	La mateixa funció segons l'estàndard POSIX: el que s'ha de complir a <i>qualsevol</i> Unix	<code>printf(3p)</code>
5 · 7	Formats de fitxer · visions generals	<code>passwd(5)</code> , <code>signal(7)</code>

Resposta. El `printf` de C és a la secció 3; `printf(1)` és l'ordre del shell. La secció 3 descriu la implementació que tens (glibc, amb les seves extensions); la 3p descriu el

contracte portable. Quan escriguis per a diversos sistemes, refia't de la 3p; quan depuris a Linux, llegeix la 3.

TD0 · 2

● Result

L'esquelet de programa

Enunciat. Cada exercici parteix d'un programa buit que inclou les capçaleres habituals i `utils.h`. Copia'l amb un nom nou per començar cada exercici.

Solució

C tds/td0/skeleton.c

```
/*
 * TD0 - skeleton.c: the empty program every exercise starts from.
 * Copy it with a new name (cp td0/skeleton.c td1/echo.c) and build with
 * `make`.
 */
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>

#include "utils.h"

int main(int argc, char *argv[])
{
    (void)argc;    /* silences -Wunused-parameter until you use them */
    (void)argv;
    return EXIT_SUCCESS;
}
```

Les línies `(void)argc;` hi són perquè compilem amb `-Wextra`, que avisa dels paràmetres no utilitzats, i amb `-Werror`, que converteix aquest avís en error. Quan el programa ja faci servir `argc` i `argv`, esborra-les.

La biblioteca `libutils` i `exit_if`

Enunciat. Llegeix `utils.h`. Què fa `exit_if` i per què fa servir `errno`? Escriu un programa que obri un fitxer i faci servir `exit_if` per informar de l'error si no existeix.

Solució

C tds/common/utils.h

```
/*
 * utils.h - small helper library shared by every lab (libutils.a).
 * Fundamentals of Operating Systems - Universidad Tecnológica Indoamerica.
 * Based on the PG204 labs (ENSEIRB-MatMeca, F. Morandat / M. Faverge).
 */
#ifndef FSO_UTILS_H
#define FSO_UTILS_H

#include <signal.h>    /* sigset_t */
#include <stddef.h>    /* size_t */
#include <sys/types.h> /* ssize_t, pid_t */

/*
 * Exit with status 1 if `condition` is true.
 * The message is `prefix: <strerror(errno)>` when errno is set, `prefix`
 otherwise.
 * errno is only meaningful right after a failed call, so use it right after
 the call you test.
 */
void exit_if(int condition, const char *prefix);

/* Write exactly `len` bytes (loops over partial writes). Returns len, or -1
 on error. */
ssize_t write_all(int fd, const void *buf, size_t len);

/* Copy everything from fd_in to fd_out until end of file (read/write loop).
 */
void transfer(int fd_in, int fd_out);

/* Naive recursive Fibonacci: a way to burn CPU time (fibo(40) takes about a
 second). */
unsigned long long fibo(int n);
```

```

/*
 * Split argv at the first "--": what comes before goes to args1, what comes
 after to args2.
 * Both arrays end with NULL. Returns -1 if a side is empty or there is no "
 --".
 */
int split_args(char *argv[], char *args1[], char *args2[]);

/* Fork a child that sleeps `seconds` seconds and exits with status 0.
 Returns its pid. */
pid_t create_waiting_child(int seconds);

/* Name of a standard signal ("SIGUSR1"), using the numbering of the running
 system. */
const char *signame(int sig);

/* Print the standard signals that belong to `set` on fd (uses write, async-
 signal-safe). */
void print_sigset(int fd, const sigset_t *set);

/* Print the pending and the blocked signals of the calling process on fd. */
void print_signals(int fd);

/* TD1, exercise 8: the structure written and read in binary by
 write_struct/read_struct. */
struct nopad {
    char c1;
    long l;
    char c2;
};

#endif /* FSO_UTILS_H */

```


C tds/td0/errors.c

```
/*
 * TD0 - errors.c: how exit_if() reports a failed system call.
 * open() fails, sets errno, and exit_if prints "open missing.txt: No such
 * file or directory".
 */
#include <fcntl.h>
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>

#include "utils.h"

int main(int argc, char *argv[])
{
    const char *path = argc > 1 ? argv[1] : "missing.txt";
    int fd = open(path, O_RDONLY);

    exit_if(fd == -1, path);          /* prefix = the file name */
    printf("%s opened as file descriptor %d\n", path, fd);
    exit_if(close(fd) == -1, "close");
    return EXIT_SUCCESS;
}
```

Sortida real

```
$ ./errors
missing.txt: No such file or directory
[exit status 1]
```

Sortida real

```
$ ./errors notes.txt
notes.txt opened as file descriptor 3
```

Quan una crida al sistema falla, retorna `-1` i desa el codi d'error (`ENOENT`, `EACCES` ...) a la variable global `errno`. `perror` converteix aquest codi en text. Per això cal cridar `exit_if` **just després** de la crida que es comprova: qualsevol crida entremig podria canviar `errno`.



Què s'ha corregit respecte de la versió heretada

La taula de noms de senyals feia servir la numeració de macOS (a Linux, `signame(SIGUSR1)` retornava un altre nom); `fibonacci` tenia tipus diferents a `utils.h` i `utils.c`; `create_waiting_child` ignorava el seu paràmetre; i les escriptures no comprovaven el resultat. Ara la taula es construeix amb les macros del sistema i una funció `write_all` repeteix les escriptures parcials.

TD0 · 4

● Result

Compilar amb `make`

Enunciat. Explica les regles principals del `Makefile` del curs: com es construeix `libutils.a`, com es compila cada exercici i com s'obtenen diverses versions del mateix programa amb macros `-D`.

Solució



Text tds/Makefile

```
# The course compiles with gcc. make's built-in default is CC=cc, which "CC
?= gcc" would not change.
ifeq ($(origin CC),default)
CC := gcc
endif
CFLAGS  ?= -O2 -g
CFLAGS += -std=gnu11 -Wall -Wextra -Werror -D_GNU_SOURCE -pthread -Icommon
LDFLAGS += -pthread
LDLIBS  += -Lbin -lutils -lm

TDS := td0 td1 td2 td3 td4 td5 td6 td7
SRCS := $(foreach t,$(TDS),$(wildcard $(t)/*.c))
BINS := $(patsubst %.c,bin/%,$(SRCS))

# Variants the lab asks to build with different macros (-D...).
VARIANTS := \
    bin/td3/buffer_1 bin/td3/buffer_2 bin/td3/buffer_3 bin/td3/buffer_4
bin/td3/buffer_5 \
    bin/td3/fork_pid_0 bin/td3/fork_pid_1 bin/td3/fork_pid_2 bin/td3/fork_pid_3
\
    bin/td4/mypipe_bug \
    bin/td5/map_none bin/td5/map_read bin/td5/map_write bin/td5/map_exec
bin/td5/map_ew \
    bin/td6/philosophers_deadlock bin/td5/shm_race bin/td6/peterson_fence

.PHONY: all test clean $(TDS) td8

all: bin/libutils.a $(BINS) $(VARIANTS) td8

$(TDS): bin/libutils.a
    @$(MAKE) --no-print-directory $(filter bin/$@/%,$(BINS) $(VARIANTS))

bin/libutils.a: common/utils.c common/utils.h
    @mkdir -p bin
    $(CC) $(CFLAGS) -c common/utils.c -o bin/utils.o
    $(AR) rcs $@ bin/utils.o
```

- `bin/libutils.a`: compila `utils.c` en un objecte i l'arxiva amb `ar`. Un fitxer `.a` és una biblioteca *estàtica*: l'enllaçador copia a cada executable només les funcions que fa servir.
- `bin/%: %.c` és una regla de patró: `bin/td1/echo` es construeix a partir de `td1/echo.c`. `$<` és el primer prerequisit i `$@`, l'objectiu.

- `-Lbin -lutils` diu a l'enllaçador on ha de buscar la biblioteca i quina ha de fer servir; `-Icommon` diu al compilador on és `utils.h`.
- `-D_GNU_SOURCE` activa totes les funcions de glibc (POSIX i extensions de Linux); `-pthread` caldrà per als fils (TD6).

TD0 · 5

● Result · sortida real

Explora el teu sistema operatiu

Enunciat. Esbrina amb ordres: quin nucli i quina distribució fas servir, quin és el primer procés (PID 1), quants mòduls del nucli hi ha carregats, quants processadors veus i quant ha durat cada etapa de l'arrencada.

Solució (sortida real a la màquina del curs)

Sortida real

```
$ uname -srm; grep PRETTY_NAME /etc/os-release; ps -p 1 -o comm=; lsmod |
tail -n +2 | wc -l; nproc; systemd-analyze
Linux 7.0.0-34-generic x86_64
PRETTY_NAME="Ubuntu 26.04 LTS"
systemd
237
28
Startup finished in 8.661s (firmware) + 3.568s (loader) + 3.253s (kernel) +
16.571s (userspace) = 32.053s
graphical.target reached after 16.569s in userspace.
```

- `uname` mostra el **nucli** (Linux 7.0); `/etc/os-release`, la **distribució** (Ubuntu 26.04): un mateix nucli serveix per a moltes distribucions.
- El PID 1 és `systemd`, el primer procés que el nucli llança en arrencar; tots els altres processos en descendeixen.
- 258 mòduls carregats: tot i que Linux és monolític, els seus controladors es carreguen i es descarreguen sobre la marxa.
- `systemd-analyze` divideix l'arrencada en firmware, carregador, nucli i espai d'usuari: el nucli és l'etapa més curta. (Pot no estar disponible a WSL o en contenidors.)

Aquests temps alimenten l'animació A1.3 · L'arrencada →

Llista de comprovació

Repte amb IA generativa · El teu propi Makefile amb proves

Objectiu. Escriu, amb l'ajuda d'una IA, un Makefile que compili cada exercici d'un laboratori en el seu propi binari i afegeixi un objectiu `test` que executi cada programa amb una entrada coneguda.

Prompt inicial. Fes-lo servir a l'eina que prefereixis (Claude, ChatGPT, Gemini, Copilot, Codex...) amb la versió *resolta* del laboratori com a context. Demana primer un pla i revisa cada pas abans d'acceptar-lo.

Prompt

Actua com a tutor de programació de sistemes en C. Tinc una carpeta amb `skeleton.c`, `utils.c` i `utils.h` (biblioteca `libutils.a`). Vull un Makefile que: (1) construeixi `libutils.a` una sola vegada, (2) compili cada `.c` de la carpeta en un executable amb `-Wall -Wextra -Werror`, (3) tingui un objectiu `test` que executi cada programa i compari la seva sortida amb un fitxer `.expected`. Primer explica les regles de patró (%) i les variables automàtiques (\$@, \$<, \$^) que faràs servir; després dona'm el Makefile comentat línia per línia.

Com comprovar el resultat

- Executa `make` dues vegades: la segona vegada no es recompila res.
- Tocar `utils.h` (`touch common/utils.h`) recompila tot el que en depèn.
- Trenca a propòsit una sortida esperada: `make test` ha de fallar.

Declara l'ús de la IA: eina, prompts rellevants, què has comprovat tu i què has corregit. El que compta és que puguis explicar i verificar cada decisió.